Clustering Languages by Dependency Graph Features

Hema Phani Sri Jonnala (220484)
Gorre Karthik (220412)
Rajarapu Jyothisha (220862)
Shreehitha Arushan Cheekoti (220310)
Taneti Naveena Rhema (221125)
Maradana Kasi Sri Roshan (220633)
Pachipenta Rohan Aditya (220741)
Gude Dayana (220414)

1 Motivation for the Research Problem

Languages differ not only in vocabulary and morphology, but also in how grammatical relations are arranged in sentences. A simple typological description classifies languages by dominant word order, such as Subject–Verb–Object (SVO), Subject–Object–Verb (SOV), and Verb–Subject–Object (VSO). However, real syntactic structure is richer than these labels: two languages with the same word-order label may differ in dependency length, head direction, adposition placement, or degree of word-order flexibility.

This project asks whether statistics computed from dependency trees can recover meaningful typological structure across languages. Dependency treebanks are useful for this question because they represent syntactic relations directly: every token is linked to a head, and these head–dependent relations encode word order, branching direction, and structural complexity. The research problem is:

Research question: Can language-level structural statistics derived from dependency trees cluster languages into meaningful typological groups?

The objective is deliberately narrow: we represent each language as a vector of dependency-based syntactic features and test whether unsupervised clustering aligns with known SVO/SOV/VSO word-order labels. A secondary objective is to determine whether the strongest signal is exact three-way word order or a broader head-final vs. head-initial distinction.

2 Hypotheses and Predictions

H1: Dependency features contain typological signal. If syntactic dependency structure reflects word-order typology, then languages with similar dominant word order should be closer in feature space than languages with different word order.

H2: SOV languages should be most separable. SOV languages are predicted to show high object-before-verb values and stronger head-final behavior, especially in noun/adposition directionality.

H3: SVO and VSO may partially overlap. Both SVO and VSO place the verb before the object and often show head-initial tendencies. Therefore, dependency features may separate SOV from non-SOV languages more clearly than they separate SVO from VSO.

These hypotheses predict that if exact typological classes are recovered, K-Means with $k = 3$ should perform best. If the broader head-final/head-initial contrast is stronger, then $k = 2$ should show better geometric clustering.

3 Methods

3.1 Data

We used Universal Dependencies treebanks in CoNLL-U format. The final analysis contains 30 languages: 16 SVO, 9 SOV, and 5 VSO. Ambiguous languages with mixed or unclear dominant word order were excluded from the final labelled evaluation. For each language, up to 5000 sentences were parsed, and sentence-level dependency statistics were averaged into one language-level feature vector.

Table 1: Final dataset composition used in the analysis.

Word-order family	Number of languages	Examples
SVO	16	English, Spanish, French, Chinese, Hebrew
SOV	9	Hindi, Japanese, Korean, Turkish, Tamil
VSO	5	Arabic, Irish, Welsh, Coptic, Ancient Hebrew
Total	30	–

3.2 Feature Extraction

For each sentence, dependency arcs were extracted from the CoNLL-U head indices. For an arc from head h to dependent d, dependency length is:

$$\ell(h, d) = |h - d|. \tag{1}$$

For each language L, mean dependency length is computed by averaging over all arcs in all sampled sentences:

$$\mathrm{MDL}(L) = \frac{1}{|S_L|} \sum_{s \in S_L} \frac{1}{|A_s|} \sum_{(h,d) \in A_s} |h - d|. \tag{2}$$

Head direction was measured using an indicator for whether the head occurs before the dependent:

$$\mathrm{HeadDir}(L) = \frac{1}{N} \sum_{(h,d)} \mathbf{1}[h < d]. \tag{3}$$

Argument-order features were computed using dependency relations whose head is a verb. For example, subject-before-verb is:

$$\mathrm{SubjBeforeV}(L) = \frac{1}{N_{subj}} \sum \mathbf{1}[d_{subj} < h_{verb}]. \tag{4}$$

The final clustering used 11 selected features: `nsubj_before_v`, `obj_before_v`, `aux_before_v`, `head_dir_ratio`, `verb_dir`, `noun_dir`, `adp_dir`, `mean_dep_len`, `mean_depth`, `mean_arity`, and `projective`. These were chosen because they directly measure argument position, head directionality, adposition direction, and basic tree structure. Noisy or redundant variables such as POS frequencies, maximum depth, maximum arity, and sentence length were excluded from the main clustering.

3.3 Inference and Evaluation

All selected features were standardized using z-score normalization. Principal Component Analysis (PCA) was used for visualization and feature interpretation. K-Means clustering was run for $k = 2, 3, 4, 5$, and hierarchical clustering and DBSCAN were used as supporting analyses. Internal cluster quality was measured using silhouette score:

$$\mathrm{Silhouette}(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}, \tag{5}$$

where $a(i)$ is the mean distance to points in the same cluster and $b(i)$ is the mean distance to the nearest other cluster. External agreement with known word-order labels was evaluated using Adjusted Rand Index (ARI), normalized mutual information (NMI), homogeneity, completeness, V-measure, and cluster purity.

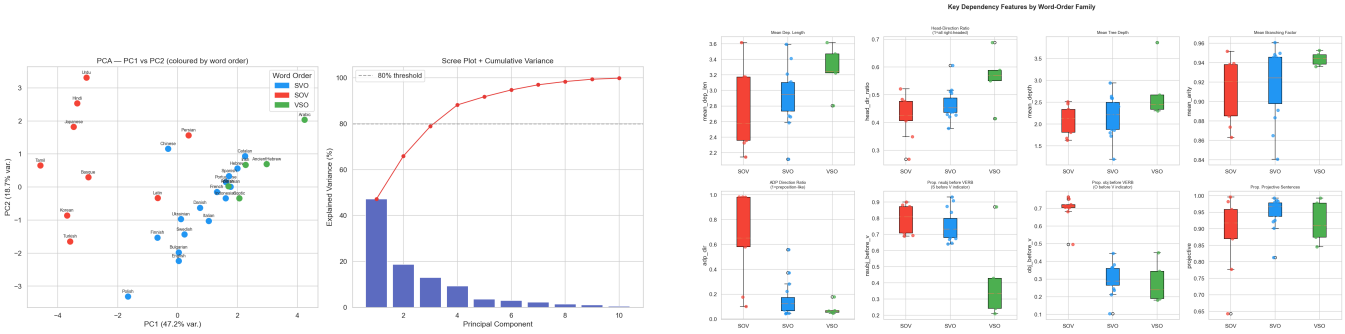
4 Results

4.1 Feature-Level Typological Patterns

The selected features show linguistically meaningful differences across word-order families. SOV languages have much higher object-before-verb values, while VSO languages have much lower subject-before-verb values. Head-direction and noun-direction features also separate head-final from head-initial tendencies.

Table 2: Mean values of selected features by word-order family. Values are averaged over languages in each class.

Feature	SOV	SVO	VSO
`nsubj_before_v`	0.793	0.752	0.417
`obj_before_v`	0.694	0.297	0.280
`aux_before_v`	0.315	0.633	0.592
`head_dir_ratio`	0.423	0.464	0.562
`verb_dir`	0.493	0.726	0.810
`noun_dir`	0.215	0.707	0.895
`adp_dir`	0.637	0.162	0.082
`mean_dep_len`	2.753	2.918	3.273
`mean_depth`	2.078	2.186	2.723
`mean_arity`	0.912	0.918	0.944
`projective`	0.883	0.950	0.920



(a) PCA biplot.

(b) Feature distributions.

Figure 1: Selected dependency features show a broad head-final vs. head-initial contrast.

The PCA analysis also supports this interpretation. PC1 mainly contrasts object-before-verb and adposition-direction behavior against noun/verb directionality and head-initial structure. Therefore, PC1 can be interpreted approximately as a head-final vs. head-initial axis. The first four principal components explain about 88% of the variance, suggesting that the selected feature set is compact and not excessively noisy.

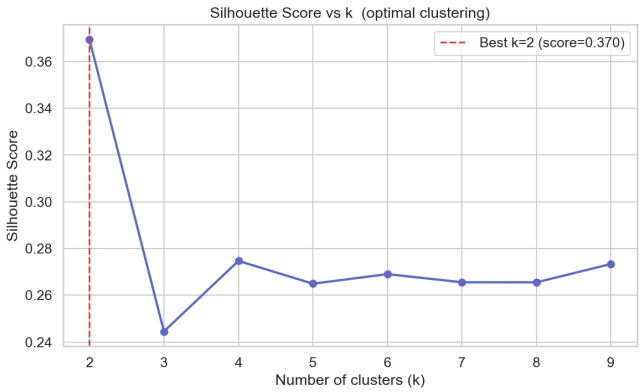
4.2 Clustering Results

K-Means clustering was tested for $k = 2, 3, 4, 5$. The best silhouette score occurs at $k = 2$, not $k = 3$.

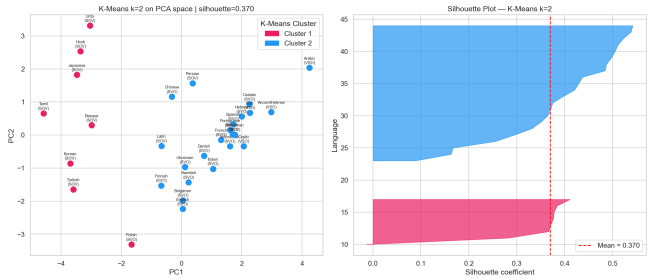
Table 3: K-Means silhouette comparison across different values of k.

k	Silhouette score
2	**0.370**
3	0.245
4	0.275
5	0.265

This result means that the strongest unsupervised structure is not the exact three-way SVO/SOV/VSO split. Instead, the model finds a broader two-way split. The $k = 2$ clustering groups strong head-final languages such as Basque, Hindi, Japanese, Korean, Tamil, Turkish, and Urdu together, while most SVO and all VSO languages fall into the other cluster. Polish is the main SVO language placed with the SOV-like cluster, while Latin and Persian are SOV languages placed with the mostly head-initial cluster.



(a) Silhouette scores.



(b) K-Means with $k = 2$.

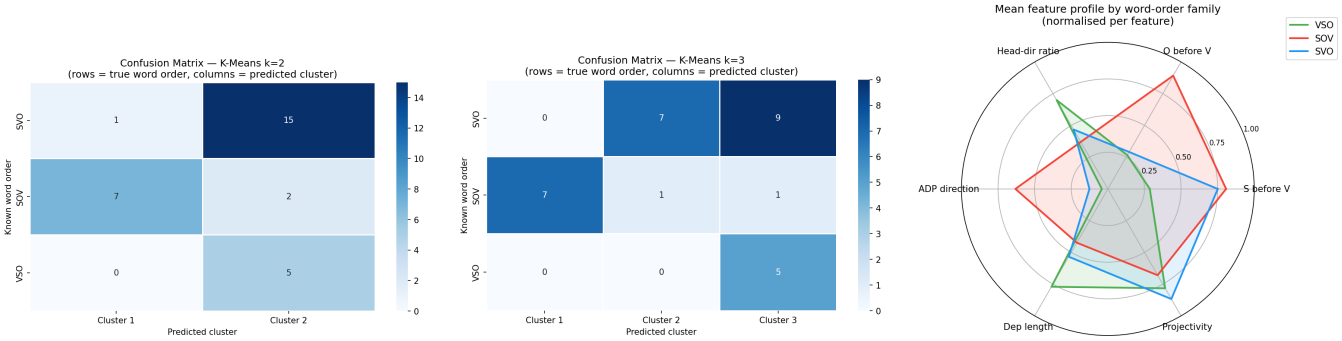
Figure 2: The best geometric clustering is obtained at $k = 2$, suggesting a head-final vs. head-initial division.

External metrics show a more nuanced picture. While $k = 2$ gives the best geometric separation, $k = 4$ gives the best agreement with labelled word-order classes on ARI, NMI, homogeneity, V-measure, and purity.

Table 4: External evaluation metrics for $k = 2, 3, 4$. Best value in each row is bold.

Metric	$k = 2$	$k = 3$	$k = 4$
Silhouette	**0.370**	0.245	0.275
ARI	0.336	0.289	**0.436**
NMI	0.376	0.460	**0.581**
Homogeneity	0.298	0.470	**0.681**
Completeness	**0.511**	0.451	0.507
V-measure	0.376	0.460	**0.581**
Cluster purity	0.733	0.767	**0.900**

Thus, $k = 2$ is the best unsupervised structural split, but $k = 4$ better approximates the labelled typological classes. The difference is important: forcing exactly three clusters is linguistically motivated, but it is not the best clustering according to either silhouette or external agreement.



(a) $k = 2$ confusion matrix.

(b) $k = 3$ confusion matrix.

(c) Mean feature radar chart.

Figure 3: Evaluation figures.

5 Theoretical Implications

The results partially support the hypothesis. Dependency-graph features do contain typological signal, but they do not recover SVO, SOV, and VSO as three perfectly separated clusters. The strongest pattern is a broader contrast between head-final and head-initial syntax. This is theoretically meaningful: word-order labels are discrete descriptions, while dependency statistics reveal gradient structural tendencies.

For theories of language processing, the results suggest that dependency relations encode processing-relevant structure beyond surface word order. Features such as object-before-verb, adposition direction, and noun/verb directionality are strong enough to group languages with similar syntactic organization. At the same time, cases like Polish, Latin, and Persian show that canonical word-order labels can be too coarse. Flexible word order, rich case marking, and mixed headedness may cause a language to behave differently in dependency-feature space than its simple typological label predicts.

For machine learning and computational typology, the main implication is that unsupervised methods can detect broad syntactic directionality, but exact typological classification requires carefully chosen features and enough balanced data. SVO and VSO languages often share head-initial properties, which explains why they are grouped together under $k = 2$. Therefore, dependency-based clustering is useful for discovering structural tendencies, but it should not be interpreted as a perfect replacement for linguistic typological analysis.

6 Conclusion

This project represented 30 languages using dependency-tree features and tested whether unsupervised clustering recovers known word-order families. The results show that the selected features capture meaningful typological structure. However, the clearest clustering is not exact SVO/SOV/VSO separation; instead, the strongest signal is a head-final vs. head-initial distinction. The work therefore supports the hypothesis in a limited but theoretically informative way: dependency structure reflects word-order typology, but the relationship is gradient rather than perfectly categorical.

References

- Chen, X. and Gerdes, K. (2017). *Classifying Languages by Dependency Structure: Typologies of Delexicalized Universal Dependency Treebanks*.
- Universal Dependencies Project. https://universaldependencies.org
- WALS Online. https://wals.info

A Implementation Details

The implementation was divided into four scripts.

1. `01_download_data_v2.py`: downloaded Universal Dependencies treebanks and saved language-level word-order metadata.
2. `02_extract_features.py`: parsed CoNLL-U files and computed dependency-based structural features for each language.
3. `03_clustering_analysis.py`: selected the final feature set, standardized features, applied PCA, t-SNE, K-Means, hierarchical clustering, and DBSCAN, and saved figures and tables.
4. `04_purity_analysis.py`: computed external evaluation metrics including ARI, NMI, homogeneity, completeness, V-measure, cluster purity, confusion matrices, and per-language assignment tables.

B Code Availability

The full code is available at: https://github.com/Roshan-633/CGS410-Project.git

C Script 1 — 01_download_datapy

Downloads Universal Dependencies treebanks for all 30 languages and saves word-order metadata.

```python
"""
Step 1: Download Universal Dependencies (UD) treebank data.
Run this first. It downloads .conllu files for ~30 languages into ./data/
"""

import os
import json
import requests
import time

DATA_DIR = "./data"
os.makedirs(DATA_DIR, exist_ok=True)


# Language  (GitHub repo name, train filename, word order)
#
# Word orders from WALS / established typological literature.
# Only using SVO / SOV / VSO  all clean, unambiguous classifications.


TREEBANKS = {

    #     SVO (head-initial, subject before verb, verb before object)
    "English":        ("UD_English-EWT",           "en_ewt-ud-train.conllu",            "SVO"),
    "Spanish":        ("UD_Spanish-GSD",            "es_gsd-ud-train.conllu",            "SVO"),
    "French":         ("UD_French-GSD",             "fr_gsd-ud-train.conllu",            "SVO"),
    "Italian":        ("UD_Italian-ISDT",           "it_isdt-ud-train.conllu",           "SVO"),
    "Portuguese":     ("UD_Portuguese-GSD",         "pt_gsd-ud-train.conllu",            "SVO"),
    "Romanian":       ("UD_Romanian-RRT",           "ro_rrt-ud-train.conllu",            "SVO"),
```

```
    "Catalan":        ("UD_Catalan-AnCora",         "ca_ancora-ud-train.conllu",         "SVO"),
    "Swedish":        ("UD_Swedish-Talbanken",      "sv_talbanken-ud-train.conllu",      "SVO"),
    "Norwegian":      ("UD_Norwegian-Bokmaal",      "nb_bokmaal-ud-train.conllu",        "SVO"),
    "Danish":         ("UD_Danish-DDT",             "da_ddt-ud-train.conllu",            "SVO"),
    "Russian":        ("UD_Russian-SynTagRus",      "ru_syntagrus-ud-train.conllu",      "SVO"),
    "Czech":          ("UD_Czech-PDT",              "cs_pdt-ud-train.conllu",            "SVO"),
    "Polish":         ("UD_Polish-LFG",             "pl_lfg-ud-train.conllu",            "SVO"),
    "Bulgarian":      ("UD_Bulgarian-BTB",          "bg_btb-ud-train.conllu",            "SVO"),
    "Ukrainian":      ("UD_Ukrainian-IU",           "uk_iu-ud-train.conllu",             "SVO"),
    "Finnish":        ("UD_Finnish-TDT",            "fi_tdt-ud-train.conllu",            "SVO"),
    "Hebrew":         ("UD_Hebrew-HTB",             "he_htb-ud-train.conllu",            "SVO"),
    "Chinese":        ("UD_Chinese-GSD",            "zh_gsd-ud-train.conllu",            "SVO"),
    "Indonesian":     ("UD_Indonesian-GSD",         "id_gsd-ud-train.conllu",            "SVO"),

    #     SOV (head-final, verb at end)
    "Turkish":        ("UD_Turkish-IMST",           "tr_imst-ud-train.conllu",           "SOV"),
    "Persian":        ("UD_Persian-Seraji",         "fa_seraji-ud-train.conllu",         "SOV"),
    "Hindi":          ("UD_Hindi-HDTB",             "hi_hdtb-ud-train.conllu",           "SOV"),
    "Japanese":       ("UD_Japanese-GSD",           "ja_gsd-ud-train.conllu",            "SOV"),
    "Korean":         ("UD_Korean-GSD",             "ko_gsd-ud-train.conllu",            "SOV"),
    "Basque":         ("UD_Basque-BDT",             "eu_bdt-ud-train.conllu",            "SOV"),
    "Latin":          ("UD_Latin-ITTB",             "la_ittb-ud-train.conllu",           "SOV"),
    "Urdu":           ("UD_Urdu-UDTB",              "ur_udtb-ud-train.conllu",           "SOV"),
    "Tamil":          ("UD_Tamil-TTB",              "ta_ttb-ud-train.conllu",            "SOV"),
    "Amharic":        ("UD_Amharic-ATT",            "am_att-ud-train.conllu",            "SOV"),

    #     VSO (verb-initial, verb before subject and object)
    "Arabic":         ("UD_Arabic-PADT",            "ar_padt-ud-train.conllu",           "VSO"),
    "Irish":          ("UD_Irish-IDT",              "ga_idt-ud-train.conllu",            "VSO"),
    "Welsh":          ("UD_Welsh-CCG",              "cy_ccg-ud-train.conllu",            "VSO"),
    "Breton":         ("UD_Breton-KEB",             "br_keb-ud-train.conllu",            "VSO"),
    "Coptic":         ("UD_Coptic-Scriptorium",     "cop_scriptorium-ud-train.conllu",   "VSO"),
```

```python
    "AncientHebrew":  ("UD_Ancient_Hebrew-PTNK",   "hbo_ptnk-ud-train.conllu",          "VSO"),

}


# Note on languages excluded deliberately:
#   German, Dutch   WALS lists both as having two dominant orders (SVO+SOV);
#                    labelling them SOV would add noise to the ground truth
#   Hungarian       WALS lists as no dominant order; too risky to label
#   Maltese         VSO in classical register but heavily mixed with SVO;
#                    ambiguous ground truth


BASE_URL = "https://raw.githubusercontent.com/UniversalDependencies"


def download_file(language, repo, filename, word_order):
    """Try master  main  dev branches."""
    save_path = os.path.join(DATA_DIR, f"{language}.conllu")

    if os.path.exists(save_path):
        size_mb = os.path.getsize(save_path) / 1e6
        print(f"  [SKIP] {language} already downloaded ({size_mb:.1f} MB).")
        return True

    for branch in ["master", "main", "dev"]:
        url = f"{BASE_URL}/{repo}/{branch}/{filename}"
        try:
            print(f"  Trying {url} ...")
            r = requests.get(url, timeout=60, stream=True)
            if r.status_code == 200:
                with open(save_path, "wb") as f:
                    for chunk in r.iter_content(chunk_size=8192):
                        f.write(chunk)
                size_mb = os.path.getsize(save_path) / 1e6
                print(f"  [OK] {language} ({word_order})  {size_mb:.1f} MB")
                return True
        except Exception as e:
            print(f"  [WARN] {branch}: {e}")
        time.sleep(0.5)

    print(f"  [FAIL] {language}  could not download from any branch.")
    return False


def save_metadata():
    meta = {lang: {"repo": repo, "file": fn, "word_order": wo}
            for lang, (repo, fn, wo) in TREEBANKS.items()}
    with open(os.path.join(DATA_DIR, "metadata.json"), "w") as f:
        json.dump(meta, f, indent=2)
    print("\n[OK] metadata.json saved.")


def print_summary(results):
    from collections import Counter
    wo_counts = Counter()
    print("\n" + "=" * 55)
    print("DOWNLOAD SUMMARY")
    print("=" * 55)
    for lang, (success, wo) in results.items():
        status = "OK  " if success else "FAIL"
```

```
        print(f"  [{status}] {lang:<20} [{wo}]")
        if success:
            wo_counts[wo] += 1

    print("\n  Final class distribution (successfully downloaded):")
    for wo in ["SVO", "SOV", "VSO"]:
        bar = "" * wo_counts[wo]
        print(f"    {wo}: {bar} {wo_counts[wo]}")
    total = sum(wo_counts.values())
    print(f"\n  Total: {total} languages")


if __name__ == "__main__":
    print("=" * 60)
    print("Downloading Universal Dependencies treebanks")
    print(f"Target: {len(TREEBANKS)} languages")
    print("=" * 60)

    results = {}
    for lang, (repo, fn, wo) in TREEBANKS.items():
        print(f"\n {lang} ({wo})")
        ok = download_file(lang, repo, fn, wo)
        results[lang] = (ok, wo)

    save_metadata()
    print_summary(results)
    print(f"\nData saved in: {os.path.abspath(DATA_DIR)}")
```

D Script 2 — `02_extract_features.py`

Parses each CoNLL-U treebank and computes 35 sentence-level dependency features, then averages them into one feature vector per language.

```
"""
Step 2: Feature Extraction from CoNLL-U treebank files.
Reads each language's .conllu file, parses dependency trees,
and computes a rich feature vector for each language.

Features computed (25 total):
     Tree-level structural features
     Head-direction features (overall + per-POS)
     Part-of-speech distribution
     Argument order indicators
     Projectivity
"""

import os
import json
import numpy as np
import pandas as pd
from collections import defaultdict

DATA_DIR  = "./data"
OUT_DIR   = "./results"
os.makedirs(OUT_DIR, exist_ok=True)

# Universal POS tags (17 tags from UD)
UPOS_TAGS = ["NOUN", "VERB", "ADJ", "ADV", "PRON", "DET", "ADP",
             "AUX",  "CCONJ","SCONJ","PART","NUM", "PROPN","PUNCT",
```

```python
             "SYM",  "X",    "INTJ"]

MAX_SENTENCES = 5000   # cap per language (speeds things up, still robust)



# CoNLL-U parser


def parse_conllu(filepath, max_sents=MAX_SENTENCES):
    """
    Generator: yields one sentence at a time as a list of token dicts.
    Skips multi-word tokens (lines like '1-2') and empty nodes ('1.1').
    """
    sentence = []
    count = 0
    with open(filepath, encoding="utf-8") as f:
        for line in f:
            line = line.rstrip("\n")
            if line.startswith("#"):
                continue
            if line == "":
                if sentence:
                    yield sentence
                    count += 1
                    if count >= max_sents:
                        return
                    sentence = []
                continue
            parts = line.split("\t")
            if len(parts) != 10:
                continue
            idx = parts[0]
            # Skip multi-word tokens and empty nodes
            if "-" in idx or "." in idx:
                continue
            try:
                token = {
                    "id":     int(parts[0]),
                    "form":   parts[1],
                    "lemma":  parts[2],
                    "upos":   parts[3],
                    "head":   int(parts[6]) if parts[6] != "_" else 0,
                    "deprel": parts[7].split(":")[0],   # use coarse deprel
                }
                sentence.append(token)
            except ValueError:
                continue



# Tree utilities


def build_children(sentence):
    """Return dict: head_id  list of child token ids."""
    children = defaultdict(list)
    for tok in sentence:
        children[tok["head"]].append(tok["id"])
    return children


```

```python
def tree_depth(tok_id, children, memo={}):
    """Recursive depth calculation with memoization."""
    if tok_id not in memo:
        if not children[tok_id]:
            memo[tok_id] = 0
        else:
            memo[tok_id] = 1 + max(tree_depth(c, children, memo)
                                   for c in children[tok_id])
    return memo[tok_id]


def get_token_depths(sentence, children):
    """Return depth of every token from the root."""
    root_id = next((t["id"] for t in sentence if t["head"] == 0), None)
    if root_id is None:
        return []
    depths = {}

    def dfs(node, d):
        depths[node] = d
        for child in children[node]:
            dfs(child, d + 1)

    dfs(root_id, 0)
    return [depths.get(t["id"], 0) for t in sentence]


def is_projective(sentence):
    """
    Check if the dependency arcs in a sentence are projective.
    An arc (ij) crosses arc (kl) if i<k<j<l or k<i<l<j.
    """
    arcs = [(t["head"], t["id"]) for t in sentence if t["head"] != 0]
    for i, (h1, d1) in enumerate(arcs):
        l1, r1 = min(h1, d1), max(h1, d1)
        for h2, d2 in arcs[i+1:]:
            l2, r2 = min(h2, d2), max(h2, d2)
            if l1 < l2 < r1 < r2 or l2 < l1 < r2 < r1:
                return False
    return True



# Feature extraction for one sentence


def extract_sentence_features(sentence):
    """
    Returns a dict of raw measurements for one sentence.
    These are later aggregated (mean/proportion) across all sentences.
    """
    n = len(sentence)
    if n < 2:
        return None

    id_to_tok = {t["id"]: t for t in sentence}
    children   = build_children(sentence)

    #     1. Dependency lengths

    dep_lengths = [abs(t["id"] - t["head"])
                   for t in sentence if t["head"] != 0]
```

```python
    mean_dep_len = np.mean(dep_lengths) if dep_lengths else 0
    std_dep_len  = np.std(dep_lengths)  if dep_lengths else 0

    #     2. Head direction (right-headed = head index < dep index)
    right_headed = [1 if t["head"] < t["id"] and t["head"] != 0 else 0
                    for t in sentence if t["head"] != 0]
    head_dir_ratio = np.mean(right_headed) if right_headed else 0.5

    # Per-POS head direction ratios
    pos_dir = defaultdict(list)
    for t in sentence:
        if t["head"] != 0:
            is_right = 1 if t["head"] < t["id"] else 0
            pos_dir[t["upos"]].append(is_right)
    adp_dir  = np.mean(pos_dir.get("ADP",  [0.5]))  # preposition vs postposition
    verb_dir = np.mean(pos_dir.get("VERB", [0.5]))
    noun_dir = np.mean(pos_dir.get("NOUN", [0.5]))
    adj_dir  = np.mean(pos_dir.get("ADJ",  [0.5]))

    #     3. Tree depth

    depths     = get_token_depths(sentence, children)
    mean_depth = np.mean(depths) if depths else 0
    max_depth  = max(depths)     if depths else 0

    #     4. Branching factor (arity)
    num_children = [len(children[t["id"]]) for t in sentence]
    mean_arity   = np.mean(num_children)
    max_arity    = max(num_children)

    #     5. POS distribution

    pos_counts = defaultdict(int)
    for t in sentence:
        pos_counts[t["upos"]] += 1
    pos_freq = {p: pos_counts[p] / n for p in UPOS_TAGS}

    #     6. Root POS

    root_is_verb = int(any(t["head"] == 0 and t["upos"] == "VERB"
                           for t in sentence))

    #     7. Argument order (SVO indicators)
    # Find verb positions; check if nsubj/obj precede or follow
    verb_positions  = {t["id"]: t["id"] for t in sentence if t["upos"] == "VERB"}
    nsubj_before_v  = []
    obj_before_v    = []
    aux_before_v    = []
    for t in sentence:
        head_tok = id_to_tok.get(t["head"])
        if head_tok and head_tok["upos"] == "VERB":
            before = 1 if t["id"] < t["head"] else 0
            if t["deprel"] == "nsubj":
                nsubj_before_v.append(before)
            elif t["deprel"] == "obj":
                obj_before_v.append(before)
            elif t["deprel"] == "aux":
                aux_before_v.append(before)

    #     8. Projectivity

    projective = int(is_projective(sentence))
```

```python

    #     9. Non-projective arc count
    arcs = [(t["head"], t["id"]) for t in sentence if t["head"] != 0]
    nonproj_count = 0
    for i, (h1, d1) in enumerate(arcs):
        l1, r1 = min(h1, d1), max(h1, d1)
        for h2, d2 in arcs[i+1:]:
            l2, r2 = min(h2, d2), max(h2, d2)
            if l1 < l2 < r1 < r2 or l2 < l1 < r2 < r1:
                nonproj_count += 1

    return {
        "mean_dep_len":     mean_dep_len,
        "std_dep_len":      std_dep_len,
        "head_dir_ratio":   head_dir_ratio,
        "adp_dir":          adp_dir,
        "verb_dir":         verb_dir,
        "noun_dir":         noun_dir,
        "adj_dir":          adj_dir,
        "mean_depth":       mean_depth,
        "max_depth":        max_depth,
        "mean_arity":       mean_arity,
        "max_arity":        max_arity,
        "root_is_verb":     root_is_verb,
        "nsubj_before_v":   np.mean(nsubj_before_v) if nsubj_before_v else 0.5,
        "obj_before_v":     np.mean(obj_before_v)   if obj_before_v   else 0.5,
        "aux_before_v":     np.mean(aux_before_v)   if aux_before_v   else 0.5,
        "projective":       projective,
        "nonproj_count":    nonproj_count,
        "sent_length":      n,
        **{f"pos_{p}": pos_freq[p] for p in UPOS_TAGS},
    }



# Aggregate across all sentences in a language


def extract_language_features(filepath, language):
    """Aggregate sentence-level features into a single language feature vector."""
    records = []
    for sent in parse_conllu(filepath):
        feats = extract_sentence_features(sent)
        if feats:
            records.append(feats)

    if not records:
        print(f"  [WARN] No records for {language}")
        return None

    df = pd.DataFrame(records)
    agg = df.mean()   # mean over all sentences
    agg["n_sentences"] = len(records)
    agg["language"]    = language
    print(f"  [OK] {language}: {len(records)} sentences, "
          f"MDL={agg['mean_dep_len']:.2f}, "
          f"HDR={agg['head_dir_ratio']:.2f}, "
          f"depth={agg['mean_depth']:.2f}")
    return agg

# Main

```

```python
def main():
    # Load metadata (word orders)
    meta_path = os.path.join(DATA_DIR, "metadata.json")
    with open(meta_path) as f:
        metadata = json.load(f)

    rows = []
    for lang in sorted(os.listdir(DATA_DIR)):
        if not lang.endswith(".conllu"):
            continue
        language = lang.replace(".conllu", "")
        filepath = os.path.join(DATA_DIR, lang)
        print(f"\nProcessing: {language}")
        feats = extract_language_features(filepath, language)
        if feats is not None:
            feats["word_order"] = metadata.get(language, {}).get("word_order", "UNK")
            rows.append(feats)

    df = pd.DataFrame(rows).set_index("language")

    # Drop non-feature columns before saving feature matrix
    info_cols = ["n_sentences", "word_order"]
    feature_df = df.drop(columns=info_cols)
    meta_df    = df[info_cols]

    feature_df.to_csv(os.path.join(OUT_DIR, "features.csv"))
    meta_df.to_csv(   os.path.join(OUT_DIR, "metadata.csv"))

    print(f"\n[DONE] Feature matrix: {feature_df.shape}")
    print(f"Features: {list(feature_df.columns)}")
    print(f"\nSaved to: {os.path.abspath(OUT_DIR)}/features.csv")


if __name__ == "__main__":
    main()
```

E Script 3 — 03_clustering_analysis.py

Applies feature selection, z-score normalization, PCA, t-SNE, K-Means (k = 2–5), hierarchical clustering, and DBSCAN. Saves all figures and the K-Means comparison table.

```python
"""
Step 3: Clustering Analysis & Visualizations
============================================
Reads features.csv + metadata.csv, then produces:

  1.  PCA biplot (PC1 vs PC2, colored by word order)
  2.  t-SNE plot (colored by word order)
  3.  K-Means clustering (k=3) + silhouette analysis
  4.  Agglomerative (Hierarchical) clustering dendrogram
  5.  DBSCAN clustering plot
  6.  Feature correlation heatmap
  7.  Feature importance (loadings) bar chart
  8.  Cluster agreement vs known typology table
  9.  Per-cluster feature means table
  10. Silhouette score vs k plot (elbow / optimal k)

All figures saved to ./results/figures/
"""

import os
import warnings
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")   # non-interactive backend for saving
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.metrics import (silhouette_score, silhouette_samples,
                             adjusted_rand_score)
from scipy.cluster.hierarchy import dendrogram, linkage
from scipy.spatial.distance import squareform

warnings.filterwarnings("ignore")

#        Paths
OUT_DIR  = "./results"
FIG_DIR  = os.path.join(OUT_DIR, "figures")
os.makedirs(FIG_DIR, exist_ok=True)

#        Colour palettes
WO_COLORS = {"SVO": "#2196F3", "SOV": "#F44336", "VSO": "#4CAF50",
             "VOS": "#FF9800", "OVS": "#9C27B0", "OSV": "#795548",
             "UNK": "#9E9E9E"}

sns.set_theme(style="whitegrid", font_scale=1.1)
FIGSIZE = (10, 7)
SELECTED_FEATURES = [
    "nsubj_before_v",
    "obj_before_v",
    "aux_before_v",
```

```python
    "head_dir_ratio",
    "verb_dir",
    "noun_dir",
    "adp_dir",
    "mean_dep_len",
    "mean_depth",
    "mean_arity",
    "projective"
]
K_VALUES_TO_TRY = [2, 3, 4, 5]


# 0.  Load data


def load_data():
    feat_path = os.path.join(OUT_DIR, "features.csv")
    meta_path = os.path.join(OUT_DIR, "metadata.csv")

    feat_df = pd.read_csv(feat_path, index_col="language")
    meta_df = pd.read_csv(meta_path, index_col="language")

    print("\n[DEBUG] features.csv shape:", feat_df.shape)
    print("[DEBUG] metadata.csv shape:", meta_df.shape)
    print("[DEBUG] Any NaN before coercion?:", feat_df.isna().sum().sum())

    feat_df = feat_df.apply(pd.to_numeric, errors="coerce")

    print("[DEBUG] Any NaN after coercion?:", feat_df.isna().sum().sum())

    if feat_df.isna().sum().sum() > 0:
        print("\n[DEBUG] NaN count by column:")
        print(feat_df.isna().sum()[feat_df.isna().sum() > 0])

        print("\n[DEBUG] Rows containing NaN:")
        print(feat_df[feat_df.isna().any(axis=1)])

    # Align indices
    langs = feat_df.index.intersection(meta_df.index)
    feat_df = feat_df.loc[langs]
    meta_df = meta_df.loc[langs]

    # Keep only selected useful features
    feat_df = feat_df[SELECTED_FEATURES]
    print(f"[INFO] Using selected features: {list(feat_df.columns)}")

    # Standardise original features
    scaler = StandardScaler()
    X_scaled_array = scaler.fit_transform(feat_df.values)
    X_scaled = pd.DataFrame(
        X_scaled_array,
        index=feat_df.index,
        columns=feat_df.columns
    )

    print("[DEBUG] Any NaN in X_scaled?:", np.isnan(X_scaled.values).sum())
    print("[DEBUG] Any +inf/-inf in X_scaled?:", np.isinf(X_scaled.values).sum())
    print("[DEBUG] X_scaled min:", np.nanmin(X_scaled.values))
    print("[DEBUG] X_scaled max:", np.nanmax(X_scaled.values))

    if np.isnan(X_scaled.values).any() or np.isinf(X_scaled.values).any():
        bad_rows, bad_cols = np.where(~np.isfinite(X_scaled.values))
```

```python
        print("\n[DEBUG] Non-finite entries in X_scaled:")
        for r, c in zip(bad_rows, bad_cols):
            print(f"  row={X_scaled.index[r]}, col={X_scaled.columns[c]}, value={X_scaled.iat[r, c]}")

    word_orders = meta_df["word_order"]
    print(f"Loaded {len(langs)} languages  {feat_df.shape[1]} features")
    return feat_df, X_scaled, word_orders, scaler


# 1.  PCA Biplot


def plot_pca(X_scaled, word_orders, feat_df):
    print("[DEBUG] plot_pca input NaN count:", np.isnan(X_scaled.values).sum())
    print("[DEBUG] plot_pca input inf count:", np.isinf(X_scaled.values).sum())

    pca   = PCA(n_components=10)
    comps = pca.fit_transform(X_scaled)
    ev    = pca.explained_variance_ratio_

    fig, axes = plt.subplots(1, 2, figsize=(16, 7))

    #     Left: scatter PC1 vs      PC2
    ax = axes[0]
    for lang, row in zip(X_scaled.index, comps):
        wo  = word_orders.get(lang, "UNK")
        col = WO_COLORS.get(wo, "#9E9E9E")
        ax.scatter(row[0], row[1], color=col, s=100, zorder=3)
        ax.annotate(lang, (row[0], row[1]),
                    fontsize=7.5, ha="center", va="bottom",
                    xytext=(0, 5), textcoords="offset points")

    legend_patches = [mpatches.Patch(color=c, label=wo)
                      for wo, c in WO_COLORS.items()
                      if wo in word_orders.values]
    ax.legend(handles=legend_patches, title="Word Order", loc="best")
    ax.set_xlabel(f"PC1 ({ev[0]*100:.1f}% var.)")
    ax.set_ylabel(f"PC2 ({ev[1]*100:.1f}% var.)")
    ax.set_title("PCA  PC1 vs PC2 (coloured by word order)")

    #     Right: scree              plot
    ax2 = axes[1]
    ax2.bar(range(1, 11), ev * 100, color="#5C6BC0")
    ax2.plot(range(1, 11), np.cumsum(ev) * 100, "o-", color="#E53935")
    ax2.set_xlabel("Principal Component")
    ax2.set_ylabel("Explained Variance (%)")
    ax2.set_title("Scree Plot + Cumulative Variance")
    ax2.axhline(80, linestyle="--", color="#9E9E9E", label="80% threshold")
    ax2.legend()

    plt.tight_layout()
    path = os.path.join(FIG_DIR, "01_pca_biplot.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"[Saved] {path}")

    # Also return PC coords and PCA object for reuse
    return pd.DataFrame(comps, index=X_scaled.index,
                        columns=[f"PC{i+1}" for i in range(10)]), pca


```

```python

# 2.  PCA Loading Plot (feature contributions)


def plot_loadings(pca, feat_df):
    loadings = pd.DataFrame(
        pca.components_[:2].T,
        index=feat_df.columns,
        columns=["PC1", "PC2"]
    )
    # Show top 15 most influential features
    top = (loadings["PC1"].abs() + loadings["PC2"].abs()).nlargest(15).index
    sub = loadings.loc[top]

    fig, ax = plt.subplots(figsize=(10, 6))
    sub[["PC1", "PC2"]].plot(kind="bar", ax=ax, color=["#5C6BC0", "#EF5350"])
    ax.set_title("Top-15 Feature Loadings on PC1 and PC2")
    ax.set_ylabel("Loading coefficient")
    ax.set_xlabel("Feature")
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    path = os.path.join(FIG_DIR, "02_pca_loadings.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"[Saved] {path}")



# 3.  t-SNE Plot


def plot_tsne(X_scaled, word_orders, pca_df):
    # Use top PCA components as input to t-SNE (reduces noise)
    n_langs = len(X_scaled)
    perplexity = min(10, n_langs - 1)

    tsne  = TSNE(n_components=2, perplexity=perplexity,
                 random_state=42, max_iter=2000)
    emb   = tsne.fit_transform(pca_df.iloc[:, :10].values)

    fig, ax = plt.subplots(figsize=FIGSIZE)
    for i, lang in enumerate(X_scaled.index):
        wo  = word_orders.get(lang, "UNK")
        col = WO_COLORS.get(wo, "#9E9E9E")
        ax.scatter(emb[i, 0], emb[i, 1], color=col, s=100, zorder=3)
        ax.annotate(lang, (emb[i, 0], emb[i, 1]),
                    fontsize=7.5, ha="center", va="bottom",
                    xytext=(0, 5), textcoords="offset points")

    legend_patches = [mpatches.Patch(color=c, label=wo)
                      for wo, c in WO_COLORS.items()
                      if wo in word_orders.values]
    ax.legend(handles=legend_patches, title="Word Order", loc="best")
    ax.set_title("t-SNE Projection (coloured by word order)")
    ax.set_xlabel("t-SNE dim 1")
    ax.set_ylabel("t-SNE dim 2")
    plt.tight_layout()
    path = os.path.join(FIG_DIR, "03_tsne.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"[Saved] {path}")

```

```python


# 4.  K-Means Clustering (k=3) + Silhouette


def plot_kmeans(X_scaled, pca_df, word_orders, feat_df, k=3):
    k_fig_dir = os.path.join(FIG_DIR, f"k_{k}")
    os.makedirs(k_fig_dir, exist_ok=True)

    km = KMeans(n_clusters=k, random_state=42, n_init=20)
    km_labels = km.fit_predict(X_scaled.values)
    sil = silhouette_score(X_scaled.values, km_labels)
    print(f"K-Means (k={k}) silhouette score: {sil:.3f}")

    cluster_colors = ["#E91E63", "#2196F3", "#4CAF50",
                      "#FF9800", "#9C27B0", "#00BCD4"]

    fig, axes = plt.subplots(1, 2, figsize=(16, 7))

    ax = axes[0]
    for i, lang in enumerate(X_scaled.index):
        c = cluster_colors[km_labels[i] % len(cluster_colors)]
        wo = word_orders.get(lang, "UNK")
        ax.scatter(pca_df.iloc[i, 0], pca_df.iloc[i, 1],
                   color=c, s=100, zorder=3)
        ax.annotate(f"{lang}\n({wo})",
                    (pca_df.iloc[i, 0], pca_df.iloc[i, 1]),
                    fontsize=7, ha="center", va="bottom",
                    xytext=(0, 5), textcoords="offset points")

    legend_patches = [
        mpatches.Patch(color=cluster_colors[j], label=f"Cluster {j+1}")
        for j in range(k)
    ]
    ax.legend(handles=legend_patches, title="K-Means Cluster")
    ax.set_xlabel("PC1")
    ax.set_ylabel("PC2")
    ax.set_title(f"K-Means k={k} on PCA space | silhouette={sil:.3f}")

    ax2 = axes[1]
    sil_vals = silhouette_samples(X_scaled.values, km_labels)
    y_lower = 10

    for j in range(k):
        vals = np.sort(sil_vals[km_labels == j])
        y_upper = y_lower + len(vals)
        ax2.fill_betweenx(np.arange(y_lower, y_upper), 0, vals,
                          color=cluster_colors[j % len(cluster_colors)],
                          alpha=0.7)
        y_lower = y_upper + 5

    ax2.axvline(sil, color="red", linestyle="--", label=f"Mean = {sil:.3f}")
    ax2.set_title(f"Silhouette Plot  K-Means k={k}")
    ax2.set_xlabel("Silhouette coefficient")
    ax2.set_ylabel("Language")
    ax2.legend()

    plt.tight_layout()
    path = os.path.join(k_fig_dir, f"04_kmeans_k{k}.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"[Saved] {path}")
```

```python

    km_series = pd.Series(km_labels, index=X_scaled.index, name="kmeans_cluster")

    # Save cluster assignments for this k
    df_out = pd.DataFrame({
        "language": km_series.index,
        "km_cluster": km_series.values + 1,
        "word_order": [word_orders.get(l, "UNK") for l in km_series.index],
    })
    df_out.to_csv(os.path.join(OUT_DIR, f"cluster_assignments_k{k}.csv"), index=False)

    # Save feature means for this k
    feat_subset = feat_df.copy()
    feat_subset["cluster"] = km_series.values + 1
    means = feat_subset.groupby("cluster").mean().round(3)
    means.to_csv(os.path.join(OUT_DIR, f"cluster_feature_means_k{k}.csv"))

    return km_series, sil


# 5.  Silhouette Score vs k  (elbow / optimal k)


def plot_silhouette_k(X_scaled):
    n_langs = len(X_scaled)
    k_range = range(2, min(n_langs, 10))
    scores  = []
    for k in k_range:
        km  = KMeans(n_clusters=k, random_state=42, n_init=20)
        lbl = km.fit_predict(X_scaled.values)
        scores.append(silhouette_score(X_scaled.values, lbl))

    fig, ax = plt.subplots(figsize=(8, 5))
    ax.plot(list(k_range), scores, "o-", color="#5C6BC0", linewidth=2)
    best_k = list(k_range)[np.argmax(scores)]
    ax.axvline(best_k, linestyle="--", color="#E53935",
               label=f"Best k={best_k} (score={max(scores):.3f})")
    ax.set_xlabel("Number of clusters (k)")
    ax.set_ylabel("Silhouette Score")
    ax.set_title("Silhouette Score vs k  (optimal clustering)")
    ax.legend()
    plt.tight_layout()
    path = os.path.join(FIG_DIR, "05_silhouette_vs_k.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"[Saved] {path}")
    print(f"  Best k = {best_k} (silhouette = {max(scores):.3f})")



# 6.  Hierarchical (Agglomerative) Clustering  Dendrogram


def plot_dendrogram(X_scaled, word_orders):
    Z = linkage(X_scaled.values, method="ward")

    # Colour leaf labels by word order
    lang_list = list(X_scaled.index)
    label_colors = [WO_COLORS.get(word_orders.get(l, "UNK"), "#9E9E9E")
                    for l in lang_list]

    fig, ax = plt.subplots(figsize=(14, 7))
```

```python
    dn = dendrogram(Z, labels=lang_list, ax=ax,
                    color_threshold=0.7 * max(Z[:, 2]),
                    above_threshold_color="#BDBDBD",
                    leaf_rotation=45)

    # Colour x-tick labels by word order
    for lbl_obj, lang in zip(ax.get_xticklabels(), dn["ivl"]):
        wo  = word_orders.get(lang, "UNK")
        lbl_obj.set_color(WO_COLORS.get(wo, "#9E9E9E"))
        lbl_obj.set_fontsize(9)

    legend_patches = [mpatches.Patch(color=c, label=wo)
                      for wo, c in WO_COLORS.items()
                      if wo in word_orders.values]
    ax.legend(handles=legend_patches, title="Word Order", loc="upper right")
    ax.set_title("Hierarchical Clustering Dendrogram (Ward linkage)\n"
                 "Leaf labels coloured by known word order")
    ax.set_ylabel("Ward distance")
    plt.tight_layout()
    path = os.path.join(FIG_DIR, "06_dendrogram.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"[Saved] {path}")



# 7.  DBSCAN Clustering


def plot_dbscan(X_scaled, pca_df, word_orders):
    # Tune eps via nearest-neighbour distances
    from sklearn.neighbors import NearestNeighbors
    nbrs = NearestNeighbors(n_neighbors=3).fit(X_scaled.values)
    dists, _ = nbrs.kneighbors(X_scaled.values)
    median_dist = np.median(dists[:, -1])

    db = DBSCAN(eps=median_dist * 1.5, min_samples=2)
    db_labels = db.fit_predict(X_scaled.values)

    unique_labels = set(db_labels)
    palette = plt.cm.tab10.colors

    fig, ax = plt.subplots(figsize=FIGSIZE)
    for i, lang in enumerate(X_scaled.index):
        lbl = db_labels[i]
        col = "#BDBDBD" if lbl == -1 else palette[lbl % len(palette)]
        ax.scatter(pca_df.iloc[i, 0], pca_df.iloc[i, 1],
                   color=col, s=100, zorder=3,
                   marker="x" if lbl == -1 else "o")
        ax.annotate(lang, (pca_df.iloc[i, 0], pca_df.iloc[i, 1]),
                    fontsize=7.5, ha="center", va="bottom",
                    xytext=(0, 5), textcoords="offset points")

    # Legend
    legend_handles = []
    for lbl in sorted(unique_labels):
        col  = "#BDBDBD" if lbl == -1 else palette[lbl % len(palette)]
        name = "Noise" if lbl == -1 else f"Cluster {lbl+1}"
        legend_handles.append(mpatches.Patch(color=col, label=name))
    ax.legend(handles=legend_handles, title="DBSCAN", loc="best")
    ax.set_title(f"DBSCAN Clustering  (eps={median_dist*1.5:.2f}, min_samples=2)")
    ax.set_xlabel("PC1"); ax.set_ylabel("PC2")
```

```python
    plt.tight_layout()
    path = os.path.join(FIG_DIR, "07_dbscan.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"[Saved] {path}")



# 8.  Feature Correlation Heatmap


def plot_correlation(feat_df):
    # Select non-POS features for clarity
    non_pos = [c for c in feat_df.columns if not c.startswith("pos_")]
    corr = feat_df[non_pos].corr()

    fig, ax = plt.subplots(figsize=(12, 10))
    sns.heatmap(corr, ax=ax, cmap="coolwarm", center=0,
                annot=True, fmt=".2f", annot_kws={"size": 7},
                linewidths=0.5, square=True)
    ax.set_title("Feature Correlation Matrix (non-POS features)")
    plt.tight_layout()
    path = os.path.join(FIG_DIR, "08_correlation_heatmap.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"[Saved] {path}")



# 9.  Key Feature Comparison by Word Order (Box plots)


def plot_feature_by_wordorder(feat_df, word_orders):
    key_features = ["mean_dep_len", "head_dir_ratio", "mean_depth",
                    "mean_arity", "adp_dir", "nsubj_before_v",
                    "obj_before_v", "projective"]
    feat_labels = {
        "mean_dep_len":    "Mean Dep. Length",
        "head_dir_ratio":  "Head-Direction Ratio\n(1=all right-headed)",
        "mean_depth":      "Mean Tree Depth",
        "mean_arity":      "Mean Branching Factor",
        "adp_dir":         "ADP Direction Ratio\n(1=preposition-like)",
        "nsubj_before_v":  "Prop. nsubj before VERB\n(S before V indicator)",
        "obj_before_v":    "Prop. obj before VERB\n(O before V indicator)",
        "projective":      "Prop. Projective Sentences",
    }

    combined = feat_df[key_features].copy()
    combined["word_order"] = word_orders

    fig, axes = plt.subplots(2, 4, figsize=(20, 10))
    axes = axes.flatten()
    palette = {wo: WO_COLORS[wo] for wo in WO_COLORS}

    for i, feat in enumerate(key_features):
        ax = axes[i]
        wo_groups = combined.groupby("word_order")[feat]
        group_data = [v.values for _, v in wo_groups]
        labels_    = [k for k, _ in wo_groups]
        bplot = ax.boxplot(group_data, patch_artist=True, labels=labels_)
        for patch, lbl in zip(bplot["boxes"], labels_):
            patch.set_facecolor(WO_COLORS.get(lbl, "#9E9E9E"))
```

```python
        # Overlay individual points
        for j, (vals, lbl) in enumerate(zip(group_data, labels_)):
            ax.scatter(np.random.normal(j + 1, 0.05, len(vals)), vals,
                       color=WO_COLORS.get(lbl, "#9E9E9E"),
                       alpha=0.7, zorder=3, s=40)
        ax.set_title(feat_labels[feat], fontsize=9)
        ax.set_ylabel(feat)

    plt.suptitle("Key Dependency Features by Word-Order Family",
                 fontsize=14, fontweight="bold", y=1.01)
    plt.tight_layout()
    path = os.path.join(FIG_DIR, "09_features_by_wordorder.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"[Saved] {path}")



# 10. Cluster Summary Table + ARI


def print_cluster_analysis(km_labels, word_orders, feat_df):
    df_out = pd.DataFrame({
        "language":   km_labels.index,
        "km_cluster": km_labels.values + 1,
        "word_order": [word_orders.get(l, "UNK") for l in km_labels.index],
    })
    df_out = df_out.sort_values(["km_cluster", "word_order"])

    print("\n" + "=" * 55)
    print("K-MEANS CLUSTER ASSIGNMENTS")
    print("=" * 55)
    for cluster_id in sorted(df_out["km_cluster"].unique()):
        rows = df_out[df_out["km_cluster"] == cluster_id]
        print(f"\n  Cluster {cluster_id}:")
        for _, row in rows.iterrows():
            print(f"    {row['language']:<20} [{row['word_order']}]")

    wo_map = {"SVO": 0, "SOV": 1, "VSO": 2}

    valid_langs = []
    true_labels = []
    pred_labels = []

    for i, lang in enumerate(km_labels.index):
        wo = word_orders.get(lang, "UNK")

        # Ignore anything not in the clean label set
        if wo not in wo_map:
            continue

        valid_langs.append(lang)
        true_labels.append(wo_map[wo])
        pred_labels.append(km_labels.iloc[i])

    if len(set(true_labels)) > 1:
        ari = adjusted_rand_score(true_labels, pred_labels)
        print(f"\n  Adjusted Rand Index (clean labels only): {ari:.3f}")
        print("  (ARI=1  perfect match, ARI=0  random, ARI<0  worse than random)")
    else:
        print("\n  ARI not computed: not enough label variety.")

```

```python
    # Per-cluster feature means
    feat_subset = feat_df[["mean_dep_len", "head_dir_ratio",
                           "mean_depth", "mean_arity",
                           "adp_dir", "nsubj_before_v"]].copy()
    feat_subset["cluster"] = km_labels.values + 1
    means = feat_subset.groupby("cluster").mean().round(3)
    print("\n  Per-cluster feature means:")
    print(means.to_string())

    # Save tables
    df_out.to_csv(os.path.join(OUT_DIR, "cluster_assignments.csv"), index=False)
    means.to_csv( os.path.join(OUT_DIR, "cluster_feature_means.csv"))
    print(f"\n  [Saved] cluster_assignments.csv, cluster_feature_means.csv")



# Main


def main():
    print("=" * 60)
    print("CLUSTERING ANALYSIS  Language Dependency Graph Features")
    print("=" * 60)

    feat_df, X_scaled, word_orders, scaler = load_data()

    print("\n[1/9] PCA biplot ...")
    pca_df, pca_obj = plot_pca(X_scaled, word_orders, feat_df)

    print("\n[2/9] PCA loadings ...")
    plot_loadings(pca_obj, feat_df)

    print("\n[3/9] t-SNE ...")
    plot_tsne(X_scaled, word_orders, pca_df)

    print("\n[4/9] K-Means clustering for multiple k values ...")

    all_k_results = []

    best_k = None
    best_score = -1
    best_labels = None

    for k in K_VALUES_TO_TRY:
        print(f"\nRunning K-Means for k={k} ...")
        km_labels_k, sil_k = plot_kmeans(X_scaled, pca_df, word_orders, feat_df, k=k)

        all_k_results.append({
            "k": k,
            "silhouette_score": sil_k
        })

        if sil_k > best_score:
            best_score = sil_k
            best_k = k
            best_labels = km_labels_k

    km_results_df = pd.DataFrame(all_k_results)
    km_results_df.to_csv(os.path.join(OUT_DIR, "kmeans_k_comparison.csv"), index=False)

    print("\nK-Means comparison:")
    print(km_results_df.to_string(index=False))
```

```
    print(f"\nBest k = {best_k} with silhouette = {best_score:.3f}")

    km_labels = best_labels

    print("\n[5/9] Silhouette vs k ...")
    plot_silhouette_k(X_scaled)

    print("\n[6/9] Hierarchical dendrogram ...")
    plot_dendrogram(X_scaled, word_orders)

    print("\n[7/9] DBSCAN ...")
    plot_dbscan(X_scaled, pca_df, word_orders)

    print("\n[8/9] Correlation heatmap ...")
    plot_correlation(feat_df)

    print("\n[9/9] Feature box-plots by word order ...")
    plot_feature_by_wordorder(feat_df, word_orders)

    print_cluster_analysis(km_labels, word_orders, feat_df)

    print(f"\n{'='*60}")
    print(f"ALL DONE  figures saved in: {os.path.abspath(FIG_DIR)}")
    print(f"{'='*60}")


if __name__ == "__main__":
    main()
```

F Script 4 — `04_purity_analysis.py`

Computes external evaluation metrics (ARI, NMI, homogeneity, completeness, V-measure, cluster purity), generates confusion matrices for k = 2 and k = 3, and produces the mean-feature radar chart.

```
"""
Step 4: Cluster Purity & Confusion Matrix Analysis
==================================================
Adds the evaluation metrics that make the results section rigorous:

  1. Cluster Purity score (standard NLP cluster evaluation)
  2. Confusion matrix: predicted cluster vs known word order
  3. Per-language assignment table with majority-vote cluster labels
  4. V-measure (homogeneity + completeness)
  5. A clean summary table suitable for copy-pasting into the report

Run AFTER 03_clustering_analysis.py
Reads:  results/cluster_assignments.csv, results/features.csv, results/metadata.csv
Writes: results/purity_analysis.csv, results/figures/10_confusion_matrix.png
        results/figures/11_radar_chart.png
"""

import os
import warnings
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
```

```python
from collections import Counter

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import (
    adjusted_rand_score,
    normalized_mutual_info_score,
    homogeneity_score,
    completeness_score,
    v_measure_score,
    confusion_matrix
)

warnings.filterwarnings("ignore")

OUT_DIR = "./results"                              # unchanged  still reads features.csv / metadata.csv from here
CSV_DIR = os.path.join(OUT_DIR, "purity")         # NEW  all CSVs go here
FIG_DIR = os.path.join(OUT_DIR, "purity-figures") # NEW  all figures go here
os.makedirs(FIG_DIR, exist_ok=True)
os.makedirs(CSV_DIR, exist_ok=True)

SELECTED_FEATURES = [
    "nsubj_before_v", "obj_before_v", "aux_before_v",
    "head_dir_ratio", "verb_dir", "noun_dir", "adp_dir",
    "mean_dep_len", "mean_depth", "mean_arity", "projective"
]

WO_COLORS = {"SVO": "#2196F3", "SOV": "#F44336", "VSO": "#4CAF50"}



# Utility: cluster purity


def cluster_purity(true_labels, pred_labels):
    """
    Purity = (1/N) * sum_k [ max_j |C_k  T_j| ]
    For each cluster, count how many belong to the majority class.
    """
    n = len(true_labels)
    total = 0
    clusters = set(pred_labels)
    for c in clusters:
        indices = [i for i, p in enumerate(pred_labels) if p == c]
        counts  = Counter(true_labels[i] for i in indices)
        total  += max(counts.values())
    return total / n



# 1. Load data and re-run clustering for k=2 and k=3 cleanly


def load_and_cluster():
    feat_df = pd.read_csv(os.path.join(OUT_DIR, "features.csv"), index_col="language")
    meta_df = pd.read_csv(os.path.join(OUT_DIR, "metadata.csv"), index_col="language")

    feat_df = feat_df.apply(pd.to_numeric, errors="coerce")
    langs   = feat_df.index.intersection(meta_df.index)
    feat_df = feat_df.loc[langs][SELECTED_FEATURES]
    meta_df = meta_df.loc[langs]
```

```python

    scaler  = StandardScaler()
    X       = pd.DataFrame(scaler.fit_transform(feat_df.values),
                           index=feat_df.index, columns=feat_df.columns)

    word_orders = meta_df["word_order"]
    return feat_df, X, word_orders



# 2. Confusion matrix plot


def plot_confusion(X, word_orders, k, suffix=""):
    km = KMeans(n_clusters=k, random_state=42, n_init=20)
    labels = km.fit_predict(X.values)

    valid_wo = ["SVO", "SOV", "VSO"]

    rows = []
    for i, lang in enumerate(X.index):
        wo = word_orders.get(lang, "UNK")
        if wo in valid_wo:
            rows.append({
                "word_order": wo,
                "cluster": labels[i]
            })

    df = pd.DataFrame(rows)

    # Build rectangular table: 3 true classes x k clusters
    cm_df = pd.crosstab(df["word_order"], df["cluster"])

    # Force all word-order rows and all cluster columns to appear
    cm_df = cm_df.reindex(index=valid_wo, columns=list(range(k)), fill_value=0)

    # Assign a majority word-order label to each cluster
    cluster_labels = []
    for c in range(k):
        col = cm_df[c]
        cluster_labels.append(col.idxmax())

    fig, ax = plt.subplots(figsize=(8, 5))
    sns.heatmap(
        cm_df,
        annot=True,
        fmt="d",
        cmap="Blues",
        xticklabels=[
            f"Cluster {i+1}"
            for i in range(k)
        ],
        yticklabels=valid_wo,
        ax=ax,
        linewidths=0.5
    )

    ax.set_xlabel("Predicted cluster")
    ax.set_ylabel("Known word order")
    ax.set_title(
        f"Confusion Matrix  K-Means k={k}\n"
        f"(rows = true word order, columns = predicted cluster)"
```

```python
    )

    plt.tight_layout()
    path = os.path.join(FIG_DIR, f"10_confusion_k{k}{suffix}.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"[Saved] {path}")

    true_raw = df["word_order"].tolist()
    pred_raw = df["cluster"].tolist()

    return labels, true_raw, pred_raw, true_raw, cluster_labels


# 3. Full metrics table


def compute_all_metrics(X, word_orders):
    valid_wo = ["SVO", "SOV", "VSO"]
    wo_map   = {w: i for i, w in enumerate(valid_wo)}
    mask     = [word_orders.get(l, "UNK") in valid_wo for l in X.index]
    true_enc = [wo_map[word_orders[l]] for l, m in zip(X.index, mask) if m]

    print("\n" + "=" * 60)
    print("FULL EVALUATION METRICS")
    print("=" * 60)
    print(f"{'Metric':<35} {'k=2':>8} {'k=3':>8} {'k=4':>8}")
    print("-" * 60)

    results = []
    for k in [2, 3, 4]:
        km     = KMeans(n_clusters=k, random_state=42, n_init=20)
        labels = km.fit_predict(X.values)
        pred_all    = labels
        pred_valid  = [labels[i] for i, m in enumerate(mask) if m]

        from sklearn.metrics import silhouette_score
        sil  = silhouette_score(X.values, pred_all)
        ari  = adjusted_rand_score(true_enc, pred_valid)
        nmi  = normalized_mutual_info_score(true_enc, pred_valid)
        hom  = homogeneity_score(true_enc, pred_valid)
        comp = completeness_score(true_enc, pred_valid)
        vm   = v_measure_score(true_enc, pred_valid)
        pur  = cluster_purity(true_enc, pred_valid)

        results.append({
            "k": k, "silhouette": sil, "ARI": ari, "NMI": nmi,
            "homogeneity": hom, "completeness": comp,
            "v_measure": vm, "purity": pur
        })

    metrics = ["silhouette", "ARI", "NMI", "homogeneity",
               "completeness", "v_measure", "purity"]
    labels_str = {
        "silhouette":    "Silhouette score",
        "ARI":           "Adjusted Rand Index",
        "NMI":           "Norm. Mutual Information",
        "homogeneity":   "Homogeneity",
        "completeness":  "Completeness",
        "v_measure":     "V-measure",
        "purity":        "Cluster Purity",
    }
```

```python
    for m in metrics:
        row = [results[i][m] for i in range(3)]
        best_idx = np.argmax(row)
        vals = []
        for i, v in enumerate(row):
            s = f"{v:.3f}"
            vals.append(s + " *" if i == best_idx else s + "  ")
        print(f"  {labels_str[m]:<33} {vals[0]:>9} {vals[1]:>9} {vals[2]:>9}")

    print("\n  (* = best for that metric)")
    print("\n  Key insight:")
    print("  - k=2 gives the best geometric separation by silhouette: broad head-final vs head-initial split")
    print("  - k=3 is linguistically motivated (SVO/SOV/VSO) but harder to separate")
    print("    because SVO and VSO share head-initial properties")
    print("  - k=4 gives the best external label agreement by ARI, NMI, V-measure, homogeneity, and purity")

    # Save to CSV
    df_metrics = pd.DataFrame(results).set_index("k")
    df_metrics.to_csv(os.path.join(CSV_DIR, "purity_analysis.csv"))
    print(f"\n[Saved] {os.path.join(OUT_DIR, 'purity_analysis.csv')}")
    return df_metrics



# 4. Per-language assignment table (for the report appendix)


def print_assignment_table(X, word_orders):
    km2 = KMeans(n_clusters=2, random_state=42, n_init=20)
    km3 = KMeans(n_clusters=3, random_state=42, n_init=20)
    l2  = km2.fit_predict(X.values)
    l3  = km3.fit_predict(X.values)

    # Label clusters by majority word order
    def label_clusters(labels, word_orders, k):
        from collections import Counter
        named = {}
        for c in range(k):
            wos = [word_orders.get(lang, "UNK")
                   for lang, lbl in zip(X.index, labels) if lbl == c]
            named[c] = Counter(wos).most_common(1)[0][0]
        return named

    labels2 = label_clusters(l2, word_orders, 2)
    labels3 = label_clusters(l3, word_orders, 3)

    print("\n" + "=" * 65)
    print("PER-LANGUAGE CLUSTER ASSIGNMENTS (for report Table)")
    print("=" * 65)
    print(f"  {'Language':<18} {'True WO':<10} {'k=2 cluster':<16} {'k=3 cluster':<16} {'k=2 correct?'}")
    print("-" * 65)

    correct_2 = 0
    total     = 0
    rows = []
    for lang, c2, c3 in zip(X.index, l2, l3):
        wo   = word_orders.get(lang, "UNK")
        cl2  = f"C{c2+1} ({labels2[c2]})"
        cl3  = f"C{c3+1} ({labels3[c3]})"
```

```
        # For k=2: head-final = SOV, head-initial = SVO+VSO
        correct = ""
        if wo in ["SVO", "VSO"] and labels2[c2] in ["SVO", "VSO"]:
            correct = "yes"
            correct_2 += 1
            total += 1
        elif wo == "SOV" and labels2[c2] == "SOV":
            correct = "yes"
            correct_2 += 1
            total += 1
        elif wo in ["SVO", "VSO", "SOV"]:
            correct = "no"
            total += 1
        print(f"  {lang:<18} {wo:<10} {cl2:<16} {cl3:<16} {correct}")
        rows.append({"language": lang, "true_word_order": wo,
                     "cluster_k2": cl2, "cluster_k3": cl3})

    if total > 0:
        print(f"\n  k=2 accuracy (head-initial vs head-final): {correct_2}/{total} = {correct_2/total:.1%}")

    df = pd.DataFrame(rows)
    df.to_csv(os.path.join(CSV_DIR, "per_language_assignments.csv"), index=False)
    print(f"[Saved] {os.path.join(OUT_DIR, 'per_language_assignments.csv')}")



# 5. Radar chart: mean features per word-order group


def plot_radar(feat_df, word_orders):
    key = ["nsubj_before_v", "obj_before_v", "head_dir_ratio",
           "adp_dir", "mean_dep_len", "projective"]
    labels = ["S before V", "O before V", "Head-dir ratio",
              "ADP direction", "Dep length", "Projectivity"]

    groups  = {}
    for lang in feat_df.index:
        wo = word_orders.get(lang, "UNK")
        if wo in ["SVO", "SOV", "VSO"]:
            if wo not in groups:
                groups[wo] = []
            groups[wo].append(feat_df.loc[lang, key].values.astype(float))

    # Normalise to [0,1] across all for radar
    all_vals = np.concatenate(list(groups.values()))
    vmin = all_vals.min(axis=0)
    vmax = all_vals.max(axis=0)
    rng  = np.where(vmax - vmin == 0, 1, vmax - vmin)

    N     = len(key)
    angles = np.linspace(0, 2 * np.pi, N, endpoint=False).tolist()
    angles += angles[:1]

    colors_radar = {"SVO": "#2196F3", "SOV": "#F44336", "VSO": "#4CAF50"}
    fig, ax = plt.subplots(figsize=(7, 7),
                           subplot_kw=dict(polar=True))

    for wo, vals_list in groups.items():
        mean_vals = np.mean(vals_list, axis=0)
        norm_vals = (mean_vals - vmin) / rng
        data = norm_vals.tolist()
```

```
        data += data[:1]
        ax.plot(angles, data, color=colors_radar[wo], linewidth=2, label=wo)
        ax.fill(angles, data, color=colors_radar[wo], alpha=0.12)

    ax.set_xticks(angles[:-1])
    ax.set_xticklabels(labels, size=10)
    ax.set_yticks([0.25, 0.5, 0.75, 1.0])
    ax.set_yticklabels(["0.25", "0.50", "0.75", "1.00"], size=8)
    ax.set_title("Mean feature profile by word-order family\n(normalised per feature)",
                 pad=20)
    ax.legend(loc="upper right", bbox_to_anchor=(1.3, 1.1))
    plt.tight_layout()
    path = os.path.join(FIG_DIR, "11_radar_chart.png")
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close()
    print(f"[Saved] {path}")



# Main


def main():
    print("=" * 60)
    print("PURITY & EVALUATION ANALYSIS")
    print("=" * 60)

    feat_df, X, word_orders = load_and_cluster()

    print("\n[1/4] Confusion matrices ...")
    plot_confusion(X, word_orders, k=2)
    plot_confusion(X, word_orders, k=3)

    print("\n[2/4] All metrics table ...")
    df_metrics = compute_all_metrics(X, word_orders)

    print("\n[3/4] Per-language assignment table ...")
    print_assignment_table(X, word_orders)

    print("\n[4/4] Radar chart ...")
    plot_radar(feat_df, word_orders)

    print(f"\n{'='*60}")
    print("DONE. New files in results/ and results/figures/")
    print(f"{'='*60}")


if __name__ == "__main__":
    main()
```